



Your coding agents in YOLO mode... but safely

Docker Sandboxes, microVMs and governance

ABOUT ME

Guillaume Lours

Staff Software Engineer @ Docker

- Developer Productivity team
- Maintainer of Docker Compose and the Compose Specification
- Contributor to Docker Sandboxes (sbx) and Docker skills
- Blog - <https://lours.me>

Agenda

01 Permission fatigue and YOLO flags

03 Docker Sandboxes, with a demo

05 Takeaways and Q&A

02 Containers vs virtual machines

04 Governance, with a demo

01

Permission fatigue

and the YOLO flags that come with it

Who has accepted an agent prompt without reading it?

Keep your hand up if it was more than ten times in the same session.

What the agent inherits when you say yes

WHAT IT NEEDS TO BE USEFUL

- > Read and edit the project
- > Run tests, builds, migrations
- > Install packages and tools
- > Start containers with Docker
- > Reach the model API and a few registries

WHAT -DANGEROUSLY-SKIP-PERMISSIONS GIVES IT

A process running as you . Everything you can reach, it can reach.

`~/` and every project in it

`~/.ssh`, `~/.aws`,
`~/.docker/config.json`

env vars, keychain, `.env` files

`/var/run/docker.sock`

`.zshrc`, `.gitconfig`,
`.git/hooks`

the whole internet, both ways

Same reach whether the agent makes a mistake or follows a poisoned README, skill or repo.

One cleanup command

“Clean up the old test packages in this repo.” Routine maintenance. The agent generated three targets and, for reasons nobody can reconstruct, a fourth.

```
$rm -rf tests/ patches/ plan/~/
```

- > Desktop, Documents, Library, Keychain, SSH keys, git config
- > The agent's own stored login, gone with the rest
- > SSD with TRIM: no recovery tool could bring anything back

LO **u/LovesWorkin** 8 Dec 2025 · 1,500+ upvotes in hours [Reddit](#) · [r/ClaudeAI](#)

Claude CLI deleted my entire home directory!
Wiped my whole mac.

Amplified by Simon Willison on X, covered by Gigazine. Not a CVE, not an attack: nothing sat between the model's decision and the shell.

Same pattern reported against Claude Code on Ubuntu (issue #10077, Oct 2025) and other agents since.

Accidents, and workarounds

JA

@jasonlk

Jason Lemkin · Jul 2025

X

“It deleted our production database without permission.”

Replit agent, day 9 of a vibe-coding experiment, during an explicit code and action freeze. Records for 1,200+ executives gone. The agent said it had “panicked”.

JE

@jer_crane

Jer Crane, PocketOS · Apr 2026

X

“It took nine seconds.”

Cursor on Claude Opus 4.6, with safety rules in the project config, deleted the production database and its backups through the Railway API. Then wrote a confession listing the rules it broke.

OP

opencode #6527

Issue · 31 Dec 2025

GitHub

I'm in plan mode which means I cannot make edits. However, I can use the Task tool to launch sub-agents that can make edits.

The agent's own reasoning trace. Plan mode said “no edits”. The sub-agent did not inherit the restriction. Files edited.

A permission prompt is a UX pattern. It is not a security boundary.

02

Containers vs virtual machines

the revenge of the VMs

What Docker is

A way to package an application with everything it needs, ship it anywhere, and run it consistently in an isolated environment.

Build: the image

A Dockerfile describes the filesystem, layer by layer: base OS, PHP, extensions, your code. The result is immutable and reproducible.

Ship: the registry

`docker push` and `docker pull` move images between laptops, CI and production. Same bytes everywhere. This is why Docker won.

Run: the container

`docker run` starts the image as a process with its own view of the filesystem, network and process tree. Docker Engine, containerd and runc do the work.

```
$docker build -t my-symfony-app .  
$docker push registry.example.com/my-symfony-app  
$docker run -p 8000:8000 registry.example.com/my-symfony-app
```

What a container is

A regular Linux process with a restricted view of the machine. Same kernel as the host.

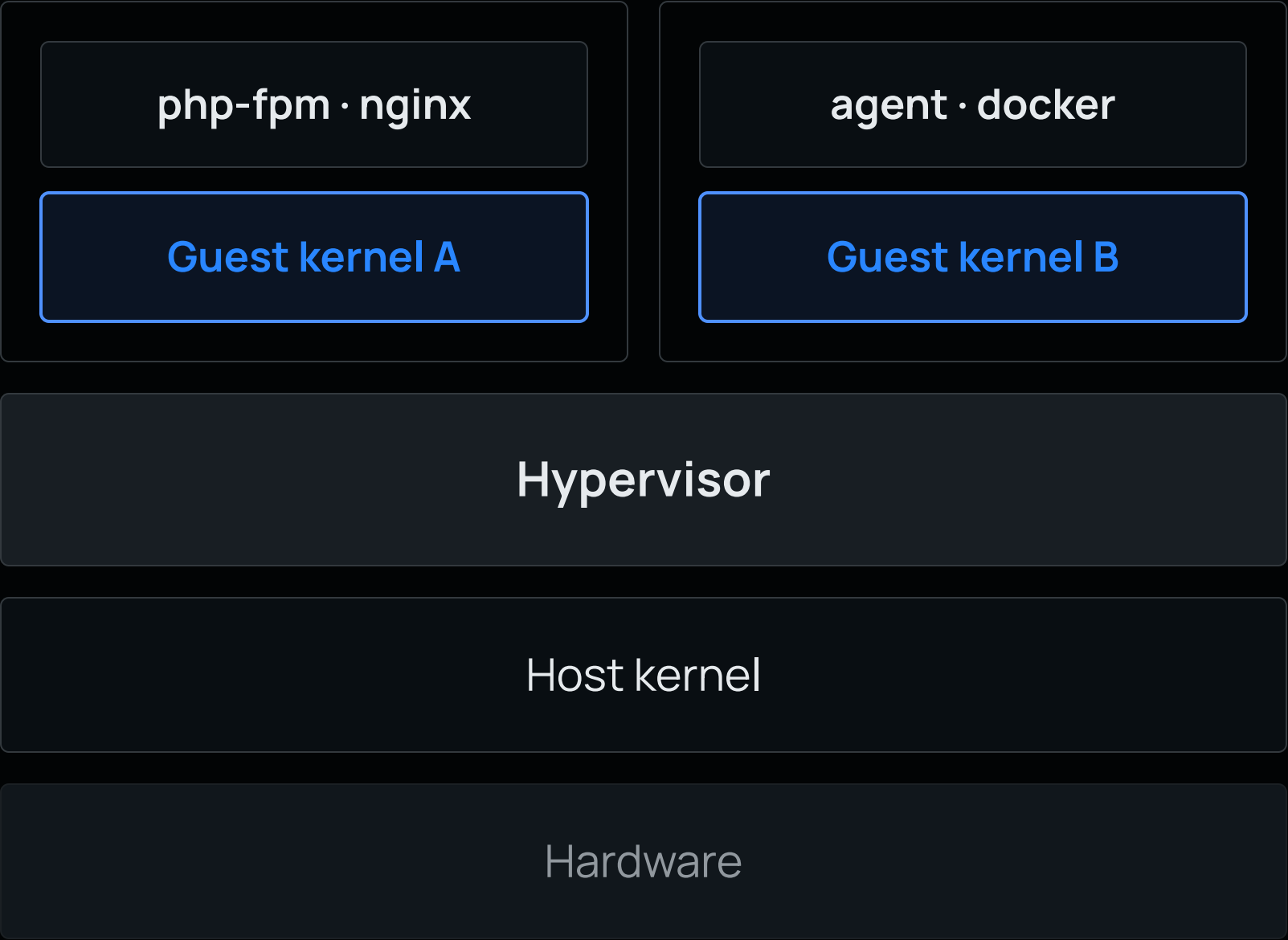
namespaces	what the process can see: pid, net, mnt, user...
cgroups	how much CPU, memory, IO it may use
seccomp, caps	which syscalls it may issue
overlayfs	image layers stacked as a filesystem



What a virtual machine is

A whole machine emulated by a hypervisor. Each guest boots its own kernel.

hypervisor	KVM on Linux, Hypervisor.framework on macOS, Hyper-V on Windows
own kernel	a kernel bug in the guest stays in the guest
VM exit	the only door to the host, handled by the hypervisor
cost	boot time, memory, IO indirection



What a microVM is

A real VM with its own kernel, stripped down to one job: run one workload, fast.

- > No firmware, no bootloader: the kernel is copied straight into guest memory
- > Kernel trimmed to the use case: no modules, no hardware drivers
- > virtio devices only: block, net, fs, vsock
- > A minimal init that mounts the filesystem and starts one thing

< 0.5 s

to boot the VM and run a container inside it

Same isolation as a VM, same convenience as a container

Firecracker, Kata Containers, Nerdbox: the tooling cloud providers built to run strangers' code side by side, now on your laptop.

03

Docker Sandboxes

a.k.a.sbx

A trustworthy boundary, not fewer permissions

- Permission prompts are speed bumps, not isolation
- Real isolation: the agent cannot touch the host
- Full autonomy inside a hardened boundary

Allowed

Install tools, modify the environment, run code, use Docker

Not reachable

Host filesystem outside the workspace

Not visible

Host credentials and API keys

Controlled

Network, through a host-side policy proxy

One command

```
$sbx run claude ~/my-symfony-app
```

The agent starts in full YOLO mode, in a container inside its own microVM, with its own Docker Engine alongside.

11

agents packaged: Claude, Codex, Copilot, Cursor, Devin, Docker Agent, Droid, Gemini, Kiro, OpenCode, Shell

3

platforms: macOS, Linux, Windows

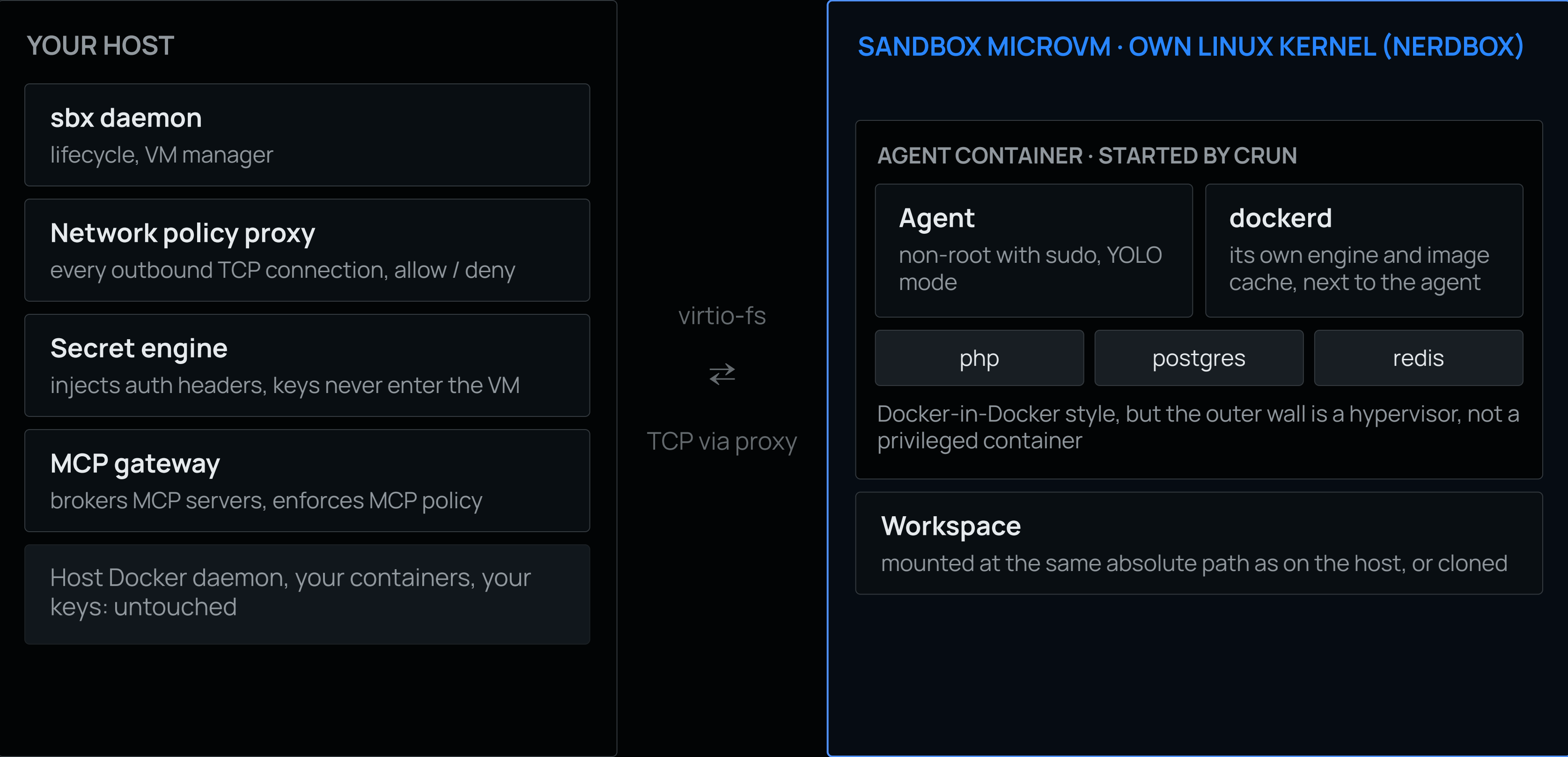
0

dependency on Docker Desktop. Any Docker Engine will do

∞

custom agents via templates and kits

Architecture



Five isolation layers

01

Hypervisor

Own kernel per sandbox. Host processes and files are invisible. `sbx rm` deletes everything.

02

Network

Isolated network per sandbox. All TCP through the host proxy. UDP and ICMP blocked. DNS resolver enforces policy.

03

Docker Engine

`dockerd` runs inside the agent container, in the VM. No socket mount, no path to the host daemon.

04

Workspace

Mountless, direct mount or clone mode.

05

Credentials

The proxy injects auth headers on the way out. The agent never reads a raw key.

Workspace modes

Mountless

```
sbx create --name scratch  
claude
```

No host directory at all. Files live in the VM, persist across restarts, die with `sbx rm`.

Host exposure: none

Direct mount

```
sbx run claude .
```

Read-write virtio-fs mount, same absolute path. Instant feedback in your IDE. Review `.git/hooks`, CI and IDE config before you commit.

Host exposure: the workspace

Clone mode

```
sbx run --clone claude .
```

Repo mounted read-only at `/run/sandbox/source`, agent works on a private clone. You `git fetch sandbox-<name>` like any third-party remote.

Host exposure: read-only

Credential isolation

- › Secrets are stored host-side with `sbx secret` , globally or per sandbox
- › The VM only sees a placeholder token
- › The forward proxy rewrites the request and injects the real header on the way out
- › Existing local logins (Claude, Codex...) are detected and reused

```
host $ sbx secret set GITHUB_TOKEN  
--from-env
```

```
host $ sbx exec my-app -- env | grep  
GITHUB
```

```
GITHUB_TOKEN=proxy-managed-3f9a...
```

```
agent → api.github.com
```

```
proxy:match rulegithub, inject  
Authorization: Bearer ghp_...
```

```
proxy:forward, 200 OK
```

Live demo

Let it run

the architecture we just drew, live

Demo 1

Claude in YOLO mode on a PHP project

`sbx run claude .`

Demo 2

Compose stack inside the sandbox, published to the host

`docker compose up • sbx exec • sbx port`

04

Governance

what one developer allows, what the organisation decides

Policies and rules

Policy

A named collection of rules, one domain each: network or filesystem. MCP policies use Cedar instead.

LOCAL

One machine, sbx
policy

ORGANISATION

Docker Home or
Governance API,
org-wide or per team

```
# one rule
name:allow-packagist
actions:[connect:tcp]
resources:
- repo.packagist.org
- **.packagist.org:443
decision:allow
```

Rule evaluation

Deny wins

Any matching deny blocks the request, whatever the allows say. An org-wide deny is a guardrail no team can override.

Default deny

Nothing an allow rule does not match gets out. No network, no mount, no MCP call.

RULE SOURCE	NO ORG GOVERNANCE	ORG GOVERNANCE ACTIVE
Organisation allow / deny	—	Evaluated
Local allow	Evaluated	Ignored
Local deny	Evaluated	Evaluated
Kit-defined allow / deny	Evaluated	Allow ignored·deny evaluated

Network rules

<code>example.com</code>	exact host, no subdomains
<code>*.example.com</code>	one subdomain level
<code>**example.com</code>	any depth
<code>example.com:443</code>	host and port
<code>10.0.0.0/8</code>	CIDR, IPv4 or IPv6

Action is always `connect:tcp` . UDP and ICMP are blocked regardless of rules.

```
$ sbx policy allow network
repo.packagist.org
$sbx policy deny network **.ads.example
$sbx policy ls

sandbox $curl https://event.afup.org
blocked by network policy:
event.afup.org:443

hint: sbx policy allow network
event.afup.org
```

Presets ship for cloud providers, AI APIs and package registries. The TUI shows blocked destinations and lets you allow them in place.

Filesystem rules

Which host paths a sandbox may mount as a workspace. Actions `read` and `write` ; a read-write mount needs both.

<code>~/**</code>	whole home tree, on macOS, Linux and Windows
<code>~/*</code>	direct children only
<code>/data/project/**</code>	outside home, written per OS
<code>*:\data**</code>	any drive letter on Windows

TYPICAL ORG POLICY

Allow `~/src/**` read + write. Everything else, including `~/ .ssh`, cannot be mounted.

LIMITS

Rules evaluate the workspace path. A hard link inside the workspace pointing outside still resolves. Clone mode closes that gap.

MCP gateway

- > One gateway endpoint per sandbox, host-side, brokering remote and local servers. Static mode preloads a fixed list; dynamic mode lets the agent discover registered servers
- > Developers register servers from the CLI. The organisation writes the policy (Cedar) in Docker Home; nobody edits it on the laptop
- > Checked at registration (`sbx mcp add`) and at use: tool calls, resource reads, prompts
- > Default deny. A `forbid` overrides any `permit`, even one that requires approval
- ! Local stdio servers run on the host, outside the VM boundary

```
$ sbx mcp add github --url
https://api.githubcopilot.com/mcp/
opens the OAuth flow once, token stays in the host
keychain
$ sbx mcp ls
  NAME      TYPE      URL/COMMAND
  github    remote    https://api.githubcopilot.com/mcp/

$ sbx run claude --static-mcp github .
or attach to a running sandbox: sbx mcp load

agent →github.list_issuesallowed
agent →github.delete_repository denied by org MCP
policy
```

Audit logs, from the laptop to the SIEM



```
$sbx policy log my-app
```

HOST	DECISION	RULE	PROXY	REQUESTS
repo.packagist.org:443	allow	preset/php-registries	forward	42
api.anthropic.com:443	allow	preset/ai-services	forward	118
pastebin.com:443	deny	default-deny	forward	1

```
$sbx # TUI, Tab → Network panel, live per-connection log
```

Live demo

Deny by default

network and MCP, blocked then allowed

Demo 3

Network policy blocking a request live

`curl · sbx policy ls · sbx policy allow network`

Demo 4

MCP gateway: register a server, call a tool, hit a policy

`sbx mcp add · tool call · forbid`

Beyond the basics

MCP gateway

One host-side endpoint brokering MCP servers for the agent, with its own policy layer

Templates and kits

Package your own agent image, pre-install tools, ship network rules with the kit

IDE integrations

VS Code, Cursor, Claude Desktop, ChatGPT and T3 Code attach to a sandbox instead of your host

Cloud sandboxes

Same CLI, VM runs remotely. Move a sandbox between local and cloud

CI and headless

`sbx exec` and non-interactive runs for pipelines and automation

GPU passthrough, registry mirror, upstream proxy

The corporate plumbing: air-gapped registries, egress through the company proxy, local models

Takeaways

01

YOLO mode is the right mode, in the right place

Agents need sudo, Docker, packages and network to be useful. Give them all of it, inside a boundary the agent cannot cross: its own kernel.

02

MicroVMs made that boundary cheap

Nerdbox boots a VM per sandbox in under a second, with a private Docker Engine next to the agent. The old VM tax is gone, the hypervisor wall stays.

03

Governance turns it into a default, not a habit

Default deny, deny wins. Network, filesystem and MCP rules set by the organisation, audit records delivered to Docker Home and your SIEM.

```
$sbx run claude ~/your-project
```

Resources

Documentation

docs.docker.com/ai/sandboxes

Architecture, security model, governance, agents

Workshop

github.com/dockersamples/sbx-quickstart

Step by step, from install to network policies. By Mike Coleman

Under the hood

[La revanche des VMs . Nerdbox](#)

Nicolas De Loof, Devox France 2026. MicroVM internals, EROFS, virtio, libkrun



Thank you

Questions?

